

Form Angular

Forms

Model Driven or Reactive Forms

model ←———— html

Template Driven Forms

- 1- Model Driven: we create model first then we map from the html to the model
 - 2- Model Driven or Reactive Forms : មានពីរសំខាន់គឺ FormGroup and FormControl
 - a. FormGroup : this is the object ជំណាង <form>
 - b. FormControl: is refrend <input> or element that we use in the <form>
- ⇒ យើងអាចធ្វើការ Validation to these two object : FromGroup and FormCiontrol
- ⇒ FormGroup is the collection of FromControl

Reactive/Model Driven Forms

First Name

Last Name

Email

Address

Street City Country

Gender

What we will do first in Reactive form or model driven forms.

3- Create form

```
4- import { FormGroup, FormControl } from '@angular/forms';
```

- FormGroup : កន្លែងដែលយើងបង្កើត model file យើង
- FormControl : កន្លែងបង្ហាញថា វាជា text, number, or drop box etc

- 1- Create FormGroup model then bind from HTML to ts
- Add ReactiveFormsModule to the app.module

```
src > app > app.module.ts > ...
1  import { NgModule } from '@angular/core';
2  import { BrowserModule } from '@angular/platform-browser';
3
4  import { AppComponent } from './app.component';
5  import { ReactiveFormsModule } from '@angular/forms';
6
7  @NgModule({
8    declarations: [
9      AppComponent
10   ],
11   imports: [
12     BrowserModule,
13     ReactiveFormsModule ✓
14   ],
15   providers: [],
16   bootstrap: [AppComponent]
17 })
18 export class AppModule { }
19
```

- 2- Bind field and html

```
src > app > app.component.html > div.container
1  <div class="container">
2    <form [formGroup]="personForm">
3
4      <div>
5        <label>First Name</label>
6        <input type="text" class="form-control" formControlName="firstName">
7      </div>
8
9    </form>
10
11 </div>
```

- formControlName: where we bind the field to the ts :
- formControlName="firstName"

⇒ firstName: new FormControl(),

- First Name :

<div>

<label>Frist Name</label>

<input type="text" class="form-control" formControlName="firstName">

</div>

- This is the first input form that we bind from html to ts

```
index.html  app.component.html  app.component.ts
src > app > app.component.html > div.container
1  <div class="container">
2    <form [formGroup]="personForm">
3
4      <div>
5        <label>Frist Name</label>
6        <input type="text" class="form-control" formControlName="firstName">
7      </div>
8
9      <div>
10       <label>Last Name</label>
11       <input type="text" class="form-control" formControlName="lastName">
12     </div>
13
14     <div>
15       <label>Email</label>
16       <input type="text" class="form-control" formControlName="email">
17     </div>
18
19     <div>
20       <label>Gender</label><br>
21       <input type="radio" value="male" formControlName="gender">Male
22       <input type="radio" value="female" formControlName="gender">Female
23     </div>
24
25   </form>
26
27 </div>
```

TS:

```
src > app > app.component.ts > AppComponent > ngOnInit
5  @Component({
6      selector: 'app-root',
7      templateUrl: './app.component.html',
8      styleUrls: ['./app.component.css']
9  })
10 export class AppComponent implements OnInit {
11
12     // Create object
13     public personForm!: FormGroup;
14
15     ngOnInit(){
16         this.personForm = new FormGroup({
17             firstName: new FormControl(),
18             lastName: new FormControl(),
19             email: new FormControl(),
20             address: new FormGroup({
21                 street: new FormControl(),
22                 city: new FormControl(),
23                 country: new FormControl()
24             }),
25             gender: new FormControl()
26         });
27     }
28
29 }
30
```

3- Sub field binding

First Name

Last Name

Email

Address

Street City Country

Gender

```

<div class="form-group" formGroupName="address">

  <div class="form-group">
    <label>Street</label>
    <input type="text" class="form-control" formControlName="street">
  </div>

  <div class="form-group">
    <label>City</label>
    <input type="text" class="form-control" formControlName="city">
  </div>

  <div class="form-group">
    <select class="form-control" formControlName="country">
      <option value="USA">USA</option>
      <option value="Canada">Canada</option>
      <option value="UK">UK</option>
    </select>
  </div>
</div>

```

- formGroupName vs formControlName

Quick Memory Trick

Imagine a family:

```

personForm
├── name
└── address
    ├── city
    └── country

```

- Family (parent) → formGroupName
- Family members (children) → formControlName

Or even shorter:

Angular Object

HTML Directive

FormGroup

formGroupName

FormControl

formControlName

Binding remember noted:

- `formGroup` = "I have the actual FormGroup object"
- `formGroupName` = **Group** (**Container**)
- `formControlName` = **Single Control** (**Input**)

Directive	Used For	Example
<code>[formGroup]</code>	Connect the main FormGroup object from TypeScript to HTML	<code><form [formGroup]="personForm"></code>
<code>formGroupName</code>	Connect a nested FormGroup inside another FormGroup	<code><div formGroupName="address"></code>
<code>formControlName</code>	Connect a single FormControl	<code><input formControlName="city"></code>

Visual Diagram

```
personForm (FormGroup)
├── name (FormControl)
└── address (FormGroup)
    ├── city (FormControl)
    └── country (FormControl)
```

HTML:

```
<> HTML

<form [formGroup]="personForm">

  <input formControlName="name">

  <div formGroupName="address">
    <input formControlName="city">
    <input formControlName="country">
  </div>

</form>
```

One-sentence memory rule

- `[formGroup]` → connect the **FormGroup** variable.
- `formGroupName` → enter a **nested group**.
- `formControlName` → connect a **single field**.

If you're ever unsure, look at your TypeScript:

- `new FormGroup(...)` → USE `formGroup` OR `formGroupName`
- `new FormControl(...)` → USE `formControlName`

4- submit button

- `<form [formGroup]="personForm" (ngSubmit)="onSubmit()">`

```
- <div>
-   <input type="submit" class="btn btn-success" value="Submit">
- </div>
```